day 19 | 251010 F

```
def bisection(function, lower_guess, upper_guess,
tolerance=2**-32):
    midpoint = (lower_guess + upper_guess)/2
    while upper_guess - lower_guess > tolerance:
        if function(lower_guess)*function(midpoint)<0:
            upper_guess = midpoint
            midpoint = (lower_guess + upper_guess)/2
        elif function(midpoint)*function(upper_guess)<0:
            lower_guess = midpoint
            midpoint = (lower_guess + upper_guess)/2
        elif function(lower_guess)*function(midpoint)>0 and
function(midpoint)*function(upper_guess)>0:
            print('no unique root in that bracket')
            break
    return(midpoint)

def newton(f, df, guess, tolerance = 2**-32):
    x = guess
    n = 0
    while abs(f(x)) > tolerance:
        x = x - f(x)/df(x)
        n += 1
    return(x, n)
```

There had previously been a problem with this secant method. I left it here just to note how difficult it was to spot this error. The problem is that we were never updating `x0`. So that left this wrong but in a difficult to spot kind of way. This could be a good error correction/error detection kind of activity, so I want to preserve it. But below that is the correct secant method code.

```
    def secant_bad(f, guess, delta, tolerance = 2**-32):
        x0 = guess
        x1 = x0 + delta
        n = 0
        while abs(f(x1))>tolerance:
            x1 = x1 - (x1-x0)/(f(x1)-f(x0))*f(x1)
            n += 1
        return(x1, n)

def secant(f, guess, delta=1, tolerance = 2**-32):
    x0 = guess
    x1 = x0 + delta
    n = 0
    while abs(f(x1))>tolerance:
        x2 = x1 - (x1-x0)/(f(x1)-f(x0))*f(x1)
        x0 = x1
```

```
        x1 = x2
        n += 1
    return(x1, n)
```

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [2]:
```
def bisection(function, lower_guess, upper_guess, tolerance=2**-32):
    midpoint = (lower_guess + upper_guess)/2
    n = 0
    while upper_guess - lower_guess > tolerance:
        if function(lower_guess)*function(midpoint)<0:
            upper_guess = midpoint
            midpoint = (lower_guess + upper_guess)/2
        elif function(midpoint)*function(upper_guess)<0:
            lower_guess = midpoint
            midpoint = (lower_guess + upper_guess)/2
        elif function(lower_guess)*function(midpoint)>0 and function(midpoir
            print('no unique root in that bracket')
            break
        n = n+1
    return(midpoint, n)

def newton(f, df, guess, tolerance = 2**-32):
    x = guess
    n = 0
    while abs(f(x)) > tolerance:
        x = x - f(x)/df(x)
        n += 1
    return(x, n)

# don't use this one!!!! #
def secant_bad(f, guess, delta, tolerance = 2**-32):
    x0 = guess
    x1 = x0 + delta
    n = 0
    while abs(f(x1))>tolerance:
        x1 = x1 - (x1-x0)/(f(x1)-f(x0))*f(x1)
        n += 1
    return(x1, n)

'''
corrected code below
'''

def secant(f, guess, delta=1, tolerance = 2**-32):
    x0 = guess
    x1 = x0 + delta
    n = 0
    while abs(f(x1))>tolerance:
        x2 = x1 - (x1-x0)/(f(x1)-f(x0))*f(x1)
        x0 = x1
        x1 = x2
```

```
        n += 1
    return(x1, n)
```

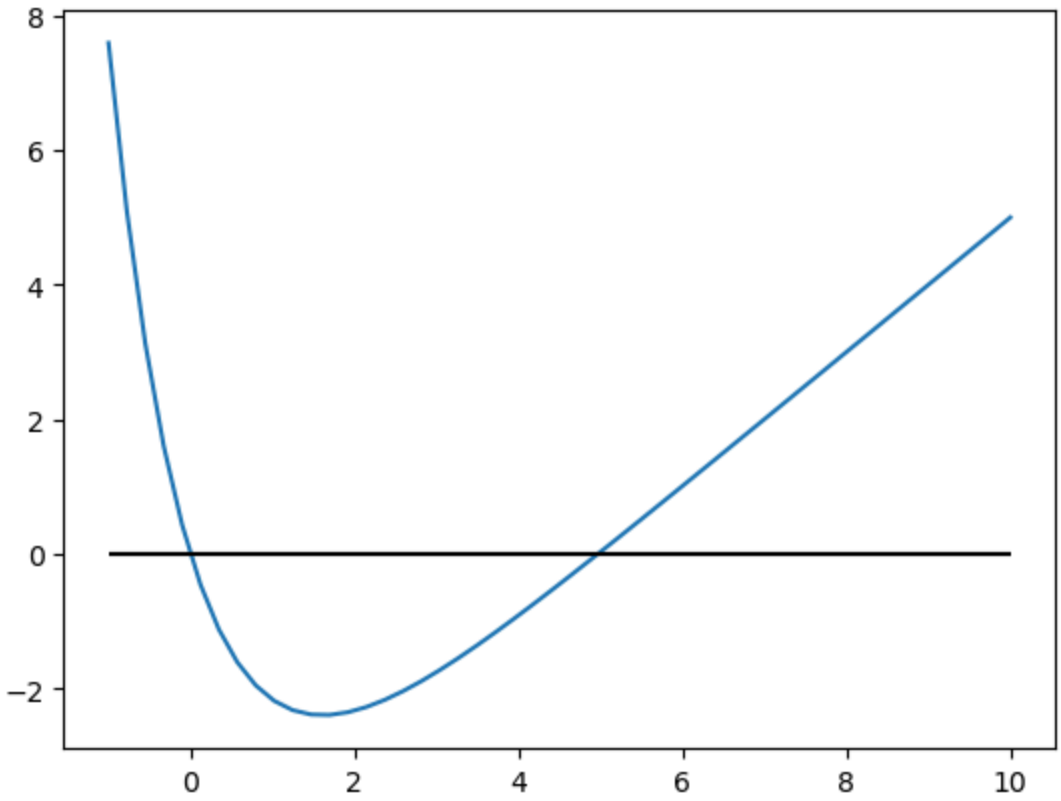
In [3]:
```
def f(x):
    return(5*np.exp(-x)+x-5)
```

In [4]:
```
fig, ax = plt.subplots()

x = np.linspace(-1,10)
y = f(x)

ax.plot(x,y)
ax.hlines(0, x[0], x[-1],'k')
```

Out[4]: <matplotlib.collections.LineCollection at 0x754784701df0>



In [5]:
```
bisection(f, 4, 6)
```

Out[5]: (4.9651142318034545, 33)

In [6]:
```
def df(x):
    return(-5*np.exp(-x)+1)
```

There are multiple different options you can try here. But the thing is that since it crosses 0 twice, you still have to be aware of what you will get.

In [9]:
```
newton(f, df, 2)
```

Out[9]: (4.965114231750363, 4)

Secant method now works even more quickly than it did before. Prior to my fix this took 7 rounds, which is what initially tipped me off that there might be a problem with it. Also teaching through it made me think there might be a problem too. But this now

In [8]:
```
secant(f, 10, 1)
```

Out[8]:
```
(4.965114231754779, 4)
```

In [ ]:
```

```